

symfoLivre — Dossier Professionnel

Titre professionnel Concepteur Développeur d'Applications (CDA) — RNCP37873

Candidat : **[Nom Prénom]**

Session : **[Date de la session d'examen]**

Organisme de formation : **[Nom de l'organisme]**

Période du projet : **[Date de début] — [Date de fin]**

i Note : Ce dossier présente un projet informatique complet réalisé de manière autonome, dans le cadre décrit en section 1.1.

Sommaire

- [symfoLivre — Dossier Professionnel](#)
 - [Sommaire](#)
 - [English Summary](#)
 - [0. Compétences couvertes](#)
 - [1. Présentation du contexte et du projet](#)
 - [1.1 Contexte de la formation et modalités de réalisation du projet](#)
 - [1.2 Origine du besoin](#)
 - [1.3 Résumé du projet](#)
 - [1.4 Cahier des charges et expression des besoins](#)
 - [1.5 Contraintes](#)
 - [2. Organisation et gestion du projet](#)
 - [2.1 Méthodologie](#)
 - [2.2 Organisation du travail](#)
 - [2.3 Outils utilisés](#)
 - [3. Architecture logicielle](#)
 - [3.1 Vue d'ensemble](#)
 - [3.2 Architecture en couches](#)
 - [3.3 Modèle MVC](#)
 - [3.4 Stratégie API](#)
 - [3.5 Architecture de sécurité — vue globale](#)
 - [4. Environnement de travail et technologies](#)
 - [4.1 Outils de conception](#)
 - [4.2 Stack technique](#)
 - [5. Conception](#)
 - [5.1 Maquettage](#)
 - [5.2 Arborescence et navigation](#)
 - [5.3 Diagramme de cas d'utilisation](#)
 - [5.4 Diagramme de classes](#)
 - [5.5 Diagrammes d'activité](#)
 - [5.6 Diagramme de déploiement](#)
 - [5.7 MCD](#)
 - [5.8 MLD](#)
 - [5.9 Script SQL / migrations](#)

- 6. Développement — cœur technique
 - 6.1 Base de données
 - 6.2 Back-end — Cas transversal : dépôt d'un livre
 - 6.3 Front-end — Cas transversal : formulaire de dépôt
 - 6.4 Panel d'administration
 - 6.5 Extrait de code significatif
- 7. Sécurité — traitement transversal
- 8. Tests
 - 8.1 Plan de test
 - 8.2 Tests unitaires
 - 8.3 Tests d'intégration et fonctionnels
 - 8.4 Résultats
 - 8.5 Audits de sécurité des dépendances
- 9. Jeu d'essai
- 10. Veille sur les vulnérabilités de sécurité
- 11. Déploiement et mise en production
 - 11.1 Préalables
 - 11.2 Étapes de déploiement
 - 11.3 Configuration serveur
 - 11.4 Hébergeur envisagé
 - 11.5 Monitoring et sauvegarde
 - 11.6 Documentation remise
 - 11.7 Diagramme de déploiement
- 12. Conclusion et bilan
 - 12.1 Fonctionnalités livrées
 - 12.2 Difficultés rencontrées et solutions apportées
 - 12.3 Axes d'amélioration identifiés
 - 12.4 Compétences mobilisées et lien avec le parcours de formation
 - Remerciements
- 13. Annexes
- Déclaration sur l'honneur

English Summary

symfoLivre is a digital library platform built with Symfony 7, allowing authors to publish books (text or audio format) and readers to browse, read online, download, and build a personal reading list. An administrator role manages categories and user accounts. The project was designed and developed entirely independently — from requirements gathering through deployment documentation and automated testing — covering the three competency blocks (CCP) of the French CDA certification: user interface design, data persistence, and multi-layer application architecture, each with an emphasis on security best practices.

0. Compétences couvertes

Compétence du référentiel	Démontrée en section
Installer et configurer son environnement de travail	\$4.2, \$6.1
Analyser les besoins et maquetter une application	\$1.4, \$5.1
Définir l'architecture logicielle	\$3
Concevoir et mettre en place une base de données relationnelle	\$5.7, \$5.8, \$5.9
Développer des composants d'accès aux données SQL	\$6.1, \$3.2
Développer des interfaces utilisateur	\$6.3, \$3.1
Développer des composants métier	\$6.2
Développer des API	\$6.2, \$3.4
Sécuriser une application à toutes les couches	\$7
Contribuer à la gestion d'un projet informatique	\$2
Préparer et documenter le déploiement	\$11
Élaborer un cahier de recettes et le mettre en œuvre	\$8, \$9

1. Présentation du contexte et du projet

1.1 Contexte de la formation et modalités de réalisation du projet

(Section importante — à ajuster avec précision selon la situation exacte et les consignes de l'organisme de formation. Voir remarque de suivi en fin de document.)

Ce projet a été réalisé dans le cadre de la formation au titre professionnel Concepteur Développeur d'Applications. Un stage a par ailleurs été effectué auprès de **[nom de l'entreprise]**, donnant lieu à une convention de stage signée et à un suivi des heures de présence en distanciel. L'entreprise d'accueil, une micro-entreprise, n'a cependant pas souhaité, pour des raisons de protection de sa propriété intellectuelle, donner accès à son code source ni confier de mission de développement encadrée.

Face à cette situation, l'initiative a été prise de définir de manière autonome un projet répondant à un besoin réel et documenté, avec la rédaction d'un cahier des charges personnel, dans le but de démontrer l'ensemble des compétences visées par le titre CDA — de la rédaction du cahier des charges jusqu'au déploiement documenté et aux tests automatisés.

1.2 Origine du besoin

Le besoin fonctionnel s'inspire des services proposés par des bibliothèques numériques spécialisées pour publics en situation de handicap visuel (à l'image du service EOLE), adaptés et simplifiés pour construire une plateforme généraliste de bibliothèque numérique couvrant l'ensemble du référentiel de compétences visé.

1.3 Résumé du projet

symfoLivre est une plateforme de bibliothèque numérique permettant à des auteurs de publier des livres (au format texte `.txt` / `.md` ou audio `.zip`), et à des lecteurs de les consulter, les lire en ligne, les télécharger et constituer un panier de lecture personnel. Un profil administrateur assure la gestion des catégories et des comptes utilisateurs. Le projet s'adresse à trois profils d'utilisateurs (lecteur, auteur, administrateur) et a été développé avec Symfony 7 et une architecture en couches sécurisée à chaque niveau.

1.4 Cahier des charges et expression des besoins

Besoins fonctionnels :

- Un visiteur peut consulter le catalogue et rechercher un livre (titre, ISBN, catégorie) sans être connecté

- Un lecteur inscrit peut lire un livre en ligne, le télécharger, et gérer un panier de lecture personnel
- Un auteur peut publier, modifier et supprimer ses propres livres
- Un administrateur peut gérer les catégories et les comptes utilisateurs (création, modification des rôles, suppression)
- Chaque utilisateur peut modifier ses informations personnelles
- Le catalogue est interrogeable via une API REST publique (recherche par titre ou ISBN)

Besoins non fonctionnels :

- Sécurité : hachage des mots de passe, protection CSRF, protection contre l'injection SQL et le XSS, contrôle d'accès par rôle
- Intégrité des données : impossibilité de supprimer un auteur ayant des livres publiés
- Ergonomie : interface responsive, cohérente avec une charte graphique dédiée
- Fiabilité : couverture par une suite de tests automatisés (unitaires, intégration, fonctionnels)
- Documentation : installation, déploiement et code documentés pour une reprise par un tiers

Livrables attendus :

- Site web public + espace authentifié (lecteur / auteur / admin)
- API REST de recherche
- Documentation d'installation (`README.md`)
- Suite de tests automatisés
- Dossier de conception (UML, MCD/MLD, maquettes)

1.5 Contraintes

- Charte graphique définie dès la phase de maquettage (voir §5.1) et respectée jusqu'au rendu final
- Formats de fichiers pris en charge limités et contrôlés : `.txt` / `.md` (texte), `.zip` (audio)
- Pas de dépendance à un service tiers payant (hébergement local de développement, base de données MariaDB auto-hébergée)

2. Organisation et gestion du projet

2.1 Méthodologie

Le projet a été mené seul, en développement itératif par fonctionnalité plutôt que selon un cadre Agile formel (Scrum/Kanban) qui suppose une équipe. La priorisation du travail s'est appuyée sur un **backlog priorisé selon la méthode MoSCoW** (Must / Should / Could / Won't), établi dès la phase de conception (voir §5), garantissant que les fonctionnalités essentielles du cahier des charges étaient développées en premier.

2.2 Organisation du travail

Le projet étant réalisé en autonomie complète, l'organisation s'est appuyée sur :

- Un développement séquencé par fonctionnalité complète (de l'entité à la vue), plutôt que par couche technique, afin de disposer à chaque étape d'une fonctionnalité testable de bout en bout
- Un outil de versioning **Git**.
- Une correction itérative pilotée par les résultats de la suite de tests automatisés (voir §8), notamment lors de la mise en place de l'environnement de test

2.3 Outils utilisés

Outil	Rôle dans le projet
Git	Gestion de versions
Symfony CLI	Serveur de développement local, commandes <code>console</code>
PlantUML	Diagrammes UML (cas d'utilisation, activité, classes)
VS Code	Environnement de développement
PHPUnit	Exécution et suivi de la suite de tests
Composer	Gestion des dépendances PHP
Firefox	Navigateur
Powershell	Console de commande

3. Architecture logicielle

3.1 Vue d'ensemble

L'application suit une architecture en couches classique d'un projet Symfony :

```

Navigateur (Twig / Bootstrap / JS)
    | HTTP
    v
Contrôleurs (src/Controller/**)
    |
    v
Repositories (src/Repository/**) ---> Entités (src/Entity/**)
    |
    v
Base de données MariaDB
```

Une couche supplémentaire, indépendante du rendu HTML, expose une **API REST JSON** (`src/Controller/Api/BookApiController.php`) consommant les mêmes repositories, illustrant la séparation entre logique métier et présentation.

3.2 Architecture en couches

Couche	Responsabilité	Technologies
Présentation	Rendu HTML, formulaires, navigation	Twig, Bootstrap 5
Contrôle	Réception des requêtes HTTP, orchestration	Symfony Controllers
Accès aux données	Requêtes, persistance	Doctrine ORM, Repositories
Domaine	Règles métier (entités, contraintes)	Entités Doctrine
Données	Stockage relationnel	MariaDB

3.3 Modèle MVC

Symfony implémente une architecture **MVC** : les entités Doctrine (Modèle) encapsulent les données et règles métier, les templates Twig (Vue) assurent le rendu, et les contrôleurs orchestrent la logique applicative entre les deux, en s'appuyant sur les services du framework (formulaires, sécurité, repositories).

3.4 Stratégie API

L'API expose des endpoints **REST en lecture seule**, au format JSON, sans authentification (recherche publique), conformément au besoin fonctionnel de permettre l'interrogation externe du catalogue :

Méthode	Route	Description
GET	/api/books?title=...	Recherche par titre (partielle)
GET	/api/books?isbn=...	Recherche par ISBN (exacte)
GET	/api/books/{id}	Détail d'un livre

3.5 Architecture de sécurité — vue globale

Rôles hiérarchisés (`ROLE_USER` < `ROLE_AUTEUR` < `ROLE_ADMIN`), authentification par session, protection CSRF sur les actions sensibles, hachage des mots de passe. Le détail complet est présenté en section 7.

4. Environnement de travail et technologies

4.1 Outils de conception

Outil	Usage	Justification
Maquettage HTML/ CSS/JS statique	Prototype interactif des 10 écrans	Contrôle total du rendu final, réutilisation directe de la charte graphique dans les templates Twig
PlantUML	Diagrammes UML (cas d'utilisation, activité, classes)	Diagrammes versionnables en texte, cohérents avec un flux de travail Git
Doctrine (MCD → MLD → entités)	Modélisation de données	Génération du schéma et des migrations directement depuis les entités PHP

4.2 Stack technique

Technologie	Rôle dans le projet	Justification du choix
PHP 8.2 / Symfony 7	Back-end, MVC, sécurité, ORM	Écosystème mature, sécurité intégrée (CSRF, hachage, RBAC), ORM Doctrine
Doctrine ORM	Accès aux données, migrations	Mapping objet-relationnel, requêtes paramétrées par défaut (protection injection SQL)
MariaDB	Base de données relationnelle	Robustesse, gratuité, compatible Doctrine
Twig	Moteur de templates	Échappement automatique des sorties (protection XSS), intégré à Symfony
Bootstrap 5	Stylisation responsive	Rapidité d'intégration, composants accessibles
PHPUnit + DAMA Doctrine Test Bundle	Tests automatisés	Standard de l'écosystème Symfony, isolation des tests par rollback transactionnel
Symfony CLI	Serveur de développement local	Rechargement rapide, intégré à l'écosystème Symfony
Git	Contrôle de version	Traçabilité, possibilité de revenir en arrière, standard professionnel

Technologie	Rôle dans le projet	Justification du choix
Composer	Gestion des dépendances PHP	Standard PHP, résolution de versions, audits de sécurité (composer audit)

5. Conception

5.1 Maquettage

Avant le développement Symfony, un prototype HTML/CSS/JS statique et interactif a été réalisé selon une démarche en deux temps : **structuration des écrans** (zoning des blocs fonctionnels : navigation, recherche, contenu, actions) puis **habillage graphique** (charte de couleurs, typographie, composants réutilisables), directement en HTML/CSS plutôt que via un outil de maquettage externe, afin de pouvoir réutiliser directement les gabarits dans les templates Twig finaux.

Une feuille de style dédiée (`css/style.css`) définit les variables de design (couleurs, rayons de bordure, ombres, transitions) réutilisées sur l'ensemble des écrans.

Écrans maquetés :

Fichier	Contenu	Accès
<code>index.html</code>	Page d'accueil : hero, barre de recherche, catalogue mis en avant	public
<code>search.html</code>	Recherche et filtrage des livres par titre / auteur / catégorie	public
<code>book.html</code>	Détail d'un livre : métadonnées, résumé, actions	public
<code>reader.html</code>	Lecture en ligne d'un livre texte	lecteur connecté
<code>basket.html</code>	Panier de lecture personnel	lecteur connecté
<code>login.html</code> / <code>register.html</code>	Authentification et inscription (choix du rôle)	public
<code>profile.html</code>	Modification des informations personnelles	utilisateur connecté
<code>my-books.html</code>	Liste des livres publiés par l'auteur connecté	auteur
<code>upload.html</code>	Formulaire de dépôt d'un livre	auteur
<code>admin.html</code>	Interface d'administration (catégories, utilisateurs)	admin

(Insérer ici une ou deux captures d'écran de la maquette, par exemple `index.html` et `upload.html`)

5.2 Arborescence et navigation

(À compléter : un schéma simple des parcours public / lecteur / auteur / admin peut être inséré ici, ou renvoyé en annexe A1.)

5.3 Diagramme de cas d'utilisation

(Renvoi Annexe A4 — un diagramme par acteur : Lecteur, Auteur, Administrateur.)

5.4 Diagramme de classes

(Renvoi Annexe A2.)

5.5 Diagrammes d'activité

Deux flux critiques ont été modélisés :

1. Parcours lecteur : recherche → consultation → lecture en ligne / téléchargement
2. Parcours auteur : dépôt d'un livre (upload) → validation → publication

(Renvoi Annexe A3.)

5.6 Diagramme de déploiement

(Renvoi Annexe A5 — voir également §11 pour le détail de la stratégie de déploiement.)

5.7 MCD

(Renvoi Annexe A6.)

5.8 MLD

(Renvoi Annexe A7.)

5.9 Script SQL / migrations

Les migrations Doctrine (`migrations/Version*.php`) constituent la source de vérité versionnée du schéma de base de données. Extrait significatif à insérer en Annexe A8.

6. Développement — cœur technique

Deux cas transversaux illustrent la chaîne complète maquette → base de données, du dépôt d'un livre par un auteur à sa recherche via l'API.

6.1 Base de données

La connexion est configurée via une URL DSN dans `.env.local` (non versionné, exclu par `.gitignore`), conformément aux bonnes pratiques (pas d'identifiants en clair dans le dépôt) :

```
DATABASE_URL="mysql://utilisateur:motdepasse@127.0.0.1:3306/symfolivre?serverVersion=mariadb-12.0.2&
```

Le schéma est généré et versionné via les migrations Doctrine (`php bin/console make:migration` puis `doctrine:migrations:migrate`), garantissant la reproductibilité de la base sur tout environnement.

6.2 Back-end — Cas transversal : dépôt d'un livre

Le contrôleur `BookController::new()` illustre la logique métier complète :

1. Réception du formulaire (`BookType`), incluant le champ fichier
2. Validation des contraintes (titre, ISBN, type MIME et taille du fichier — voir §7)
3. Déplacement du fichier uploadé vers `uploads/books/` avec un nom généré aléatoirement (protection contre l'écrasement et la divulgation du nom de fichier original)
4. Persistance de l'entité `Book` en base (Doctrine : `persist` / `flush`)
5. Redirection avec message de confirmation

(Extrait de code à insérer en Annexe A9/A10 — ≤ 15 lignes dans le corps si un extrait doit être montré ici.)

6.3 Front-end — Cas transversal : formulaire de dépôt

Le template `book/_form.html.twig` illustre l'intégration Twig + Bootstrap :

- Rendu des champs via les helpers Symfony Form (`form_widget` , `form_errors`) avec classes Bootstrap
- Attribut `accept=".txt,.md,.zip"` sur le champ fichier pour guider l'utilisateur côté client (la validation réelle reste serveur, voir §7)
- Composant réutilisé à l'identique pour la création et la modification d'un livre

6.4 Panel d'administration

L'interface `/admin/users` (`src/Controller/Admin/UserController.php`) fournit un CRUD complet de gestion des comptes utilisateurs et de leurs rôles, avec une règle métier explicite empêchant la suppression d'un auteur ayant des livres publiés.

6.5 Extrait de code significatif

(1 à 2 extraits commentés maximum dans le corps du texte, le reste en Annexe A9-A13.)

7. Sécurité — traitement transversal

La sécurité n'a pas été traitée comme un module isolé mais intégrée à chaque couche du projet.

Mesure	Détail	Fichiers
Hachage des mots de passe	bcrypt via <code>UserPasswordHasherInterface</code>	<code>src/Controller/SecurityController.php</code>
Authentification	Session, formulaire de connexion sécurisé	<code>config/packages/security.yaml</code>
Autorisation / RBAC	Rôles hiérarchisés, <code>access_control</code> déclaratif par route, vérifications de propriété en contrôleur	<code>config/packages/security.yaml</code>
CSRF	Jeton systématique sur toute action de modification/suppression	<code>templates/**/_delete_form.html.twig</code>
XSS	Échappement automatique des sorties Twig	<code>templates/**</code>
Injection SQL	Requêtes 100 % paramétrées via Doctrine QueryBuilder	<code>src/Repository/**</code>
Validation des entrées	Contraintes Symfony Validator sur les entités et formulaires	<code>src/Entity/**</code> , <code>src/Form/**</code>
Variables sensibles	<code>.env.local</code> non versionné, exclu du dépôt Git	<code>.gitignore</code>
Contrôle d'accès API	Endpoints publics en lecture seule, aucune donnée sensible exposée	<code>src/Controller/Api/BookApiController.php</code>
Audits dépendances	<code>composer audit</code>	(résultat à insérer — voir §10)

Axe d'amélioration identifié : les vérifications de propriété sont actuellement réalisées par des conditions explicites dans les contrôleurs plutôt que par des Voters Symfony dédiés — évolution naturelle pour centraliser cette logique.

8. Tests

8.1 Plan de test

Élément	Détail
Objectif	Garantir le comportement fonctionnel et la robustesse des règles de sécurité et métier
Portée	Entités, repositories, contrôleurs (HTTP), API
Stratégie	Pyramide à trois niveaux : unitaire → intégration → fonctionnel
Environnement	Base de données de test isolée (<code>symfonylivre_test</code>), rollback automatique par test (DAMA Doctrine Test Bundle)
Critères d'acceptation	100 % des tests passent avant toute livraison

8.2 Tests unitaires

Logique métier isolée, sans dépendance externe (ex. : rôles par défaut d'un utilisateur). Voir `tests/Entity/UserTest.php` , code complet en Annexe A13.

8.3 Tests d'intégration et fonctionnels

- Intégration : requêtes Doctrine contre une base réelle (`tests/Repository/BookRepositoryTest.php`)
- Fonctionnels : parcours HTTP complets — authentification, contrôle d'accès par rôle, CSRF, règles métier, API (`tests/Controller/**`)

8.4 Résultats

Niveau	Nombre de tests	Résultat
Unitaire	4	☑
Intégration	3	☑
Fonctionnel	14	☑
Total	21	☑ 100 %

PHPUnit 11.5.56 by Sebastian Bergmann and contributors.

21 / 21 (100%)

OK (21 tests, 38 assertions)

8.5 Audits de sécurité des dépendances

(À compléter avant soumission : exécuter `composer audit` sur le projet réel et insérer le résultat ci-dessous.)

[À insérer : sortie de la commande `composer audit`]

9. Jeu d'essai

Scénarios de validation manuelle, en complément de la suite de tests automatisés.

Scénario	Données d'entrée	Résultat attendu	Résultat obtenu
Connexion réussie	Email + mot de passe valides	Redirection vers l'accueil, session ouverte	☑ Conforme
Connexion échouée	Email inconnu ou mot de passe invalide	Message d'erreur, reste sur la page de connexion	☑ Conforme
Dépôt d'un livre par un auteur	Fichier <code>.txt</code> valide, titre et ISBN renseignés	Livre créé, visible dans <code>/my-books</code> et <code>/book</code>	☑ Conforme
Téléchargement d'un livre	Clic sur « Télécharger »	Fichier téléchargé, nommé <code>{Titre}.txt</code> , contenu identique à l'original	☑ Conforme
Suppression d'un auteur ayant des livres (admin)	Tentative de suppression via <code>/admin/users</code>	Blocage, message d'erreur explicite	☑ Conforme (couvert par test automatisé)
Recherche API par ISBN exact	<code>GET /api/books?isbn=...</code>	Réponse JSON avec le livre correspondant	☑ Conforme (bug corrigé, voir §12.2)
Dépôt d'un fichier de type non autorisé	Fichier <code>.exe</code>	Rejet du formulaire avec message d'erreur	⚠ À revalider après réactivation de la contrainte de validation (voir §12.2)

(Captures d'écran des scénarios à insérer en Annexe A14.)

10. Veille sur les vulnérabilités de sécurité

(Section à compléter avec les actions réellement menées.)

Le processus de veille repose sur :

- L'exécution périodique de `composer audit` pour détecter les dépendances présentant des vulnérabilités connues (base CVE)
- Le suivi des annonces de sécurité officielles Symfony (symfony.com/blog/category/security-advisories)
- La mise à jour régulière des dépendances via `composer outdated` puis `composer update` ciblé

(Insérer ici, le cas échéant, une vulnérabilité détectée et la correction apportée — même mineure, cela démontre la mise en pratique de la veille.)

11. Déploiement et mise en production

Le déploiement n'a pas été exécuté sur un serveur de production réel dans le cadre de ce projet personnel ; la démarche est néanmoins documentée ci-dessous, conformément au processus qui serait suivi.

11.1 Préalables

Serveur disposant de PHP 8.2+, d'une extension `pdo_mysql`, d'un accès SSH, d'un nom de domaine et d'un certificat HTTPS (Let's Encrypt).

11.2 Étapes de déploiement

```
git clone <depot>
cd symfoLivre
composer install --no-dev --optimize-autoloader
cp .env .env.prod.local # puis renseigner DATABASE_URL, APP_SECRET
php bin/console doctrine:migrations:migrate --no-interaction --env=prod
php bin/console cache:clear --env=prod
chmod -R 775 var/
```

11.3 Configuration serveur

Serveur web (Nginx ou Apache) configuré en reverse proxy vers PHP-FPM, document root pointé sur `public/`, utilisateur MariaDB dédié à l'application (pas de compte `root` en production).

11.4 Hébergeur envisagé

(À compléter selon le choix réel ou hypothétique, ex. : o2switch, OVH.)

11.5 Monitoring et sauvegarde

Journalisation via les logs Symfony (`var/log/prod.log`), sauvegardes régulières de la base de données (`mysqldump` planifié), supervision de disponibilité recommandée (ex. : UptimeRobot).

11.6 Documentation remise

Le fichier `README.md` du dépôt sert de documentation d'installation et de déploiement de référence.

11.7 Diagramme de déploiement

(Renvoi Annexe A5.)

12. Conclusion et bilan

12.1 Fonctionnalités livrées

- Gestion complète des livres (CRUD, upload, lecture en ligne, téléchargement)
- Gestion des catégories (CRUD, réservé aux administrateurs)
- Panier de lecture personnel
- Authentification, inscription avec choix de rôle, gestion de profil personnel
- Administration des comptes utilisateurs et des rôles
- API REST de recherche publique (titre / ISBN / identifiant)
- Suite de tests automatisés (21 tests, 3 niveaux de couverture)

12.2 Difficultés rencontrées et solutions apportées

Configuration de l'environnement de test. La mise en place de PHPUnit a nécessité plusieurs itérations : conflit de nommage de base de données (le `dbname_suffix` de Doctrine ajoutant automatiquement le suffixe `_test`), droits insuffisants de l'utilisateur MariaDB applicatif, puis erreur de namespace lors de l'enregistrement du bundle DAMA Doctrine Test Bundle. Chaque cause a été isolée méthodiquement par lecture des messages d'erreur et vérification directe du code source du package installé.

Bug détecté par les tests automatisés. La suite de tests a révélé un bug réel dans la méthode `search()` du `BookRepository` : le paramètre de requête utilisé pour la comparaison exacte sur l'ISBN (opérateur `=`) réutilisait par erreur la même valeur avec jokers (`% ... %`) que la recherche partielle par titre (opérateur `LIKE`), rendant la recherche par ISBN exact non fonctionnelle.

```
php :
// Avant (bug) :
$qb->setParameter('q', '%' . $query . '%'); // utilise aussi pour l'egalite stricte

// Apres (correction) :
$qb->setParameter('q', '%' . $query . '%')      // pour LIKE (titre, categorie)
->setParameter('isbn', $query);                  // pour = (ISBN exact)
```

Tests fonctionnels et jetons CSRF. Les premiers tests fonctionnels sur les suppressions protégées par CSRF généraient manuellement un jeton en dehors de tout contexte de requête HTTP, provoquant une erreur d'absence de session. La correction a consisté à extraire le jeton directement du formulaire HTML réellement rendu par la page.

Validation des fichiers uploadés. Une vérification a posteriori a révélé que la contrainte de validation du champ fichier (type MIME et taille) était commentée dans le formulaire, permettant l'upload de n'importe quel type de fichier sans restriction. Correction apportée en réactivant la contrainte `File` de Symfony Validator.

12.3 Axes d'amélioration identifiés

- Migration des contrôles d'autorisation métier vers des Voters Symfony dédiés
- Pagination et tri des résultats de recherche
- Mise en place d'une intégration continue (CI) exécutant la suite de tests à chaque commit
- Déploiement réel sur un environnement de production

12.4 Compétences mobilisées et lien avec le parcours de formation

Ce projet a permis de mobiliser l'ensemble des compétences du référentiel CDA : de la conception (UML, modélisation de données, maquettage) au développement (Symfony, Doctrine, API REST), en passant par la sécurisation à chaque couche et la validation par des tests automatisés — démontrant une maîtrise du cycle de développement complet d'une application web sécurisée, réalisée en totale autonomie faute d'accès au code de l'entreprise d'accueil.

Remerciements

(À compléter : formateurs, tuteur pédagogique, entourage.)

13. Annexes

N°	Contenu
A1	Arborescence / parcours utilisateurs
A2	Diagramme de classes UML
A3	Diagrammes d'activités UML (parcours lecteur, parcours auteur)
A4	Diagrammes de cas d'utilisation (Lecteur, Auteur, Administrateur)
A5	Diagramme de déploiement
A6	MCD
A7	MLD
A8	Extrait du script SQL / migrations
A9	Code : entité <code>Book</code>
A10	Code : <code>BookController</code> (dépôt d'un livre)
A11	Code : <code>Admin/UserController</code>
A12	Code : <code>BookApiController</code>
A13	Code : suite de tests PHPUnit
A14	Captures d'écran : maquette, interface publique, panel admin, jeu d'essai
A15	Résultat <code>composer audit</code>
A16	<code>README.md</code> — documentation d'installation et de déploiement

Déclaration sur l'honneur

(Modèle indicatif — reprendre le formulaire officiel fourni par l'organisme de formation ou le certificateur, s'il en existe un.)

Je soussigné(e) **[Nom Prénom]** déclare sur l'honneur que le projet présenté dans ce dossier a été réalisé personnellement, dans les conditions décrites en section 1.1, et que l'ensemble des éléments présentés reflète fidèlement mon travail.

Fait à **[Ville]**, le **[Date]**.

Signature :